UNIVERSITÄT SALZBURG

# Complete Symbolic Analysis in Selfie

Presenter: Alireza Abyaneh

This is a project in Computational Systems Group at the University of Salzburg (github.com/cksystemsgroup)

# Problem:

C

```
...;
c = c * 2;
if (c / 8 == 31)
  ...;
```

compile

RISC-V

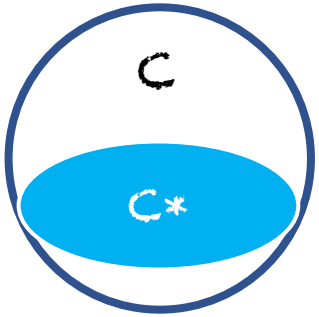
```
ld   $t0,-8($fp)
addi $t1,$zero,2
mul  $t0,$t0,$t1
sd   $t0,-8($fp)
ld   $t0,-8($fp)
addi $t1,$zero,8
divu $t0,$t0,$t1
addi $t1,$zero,31
sub  $t0,$t1,$t0
addi $t1,$zero,1
sltu $t0,$t0,$t1
beq  $t0,$zero,...
```

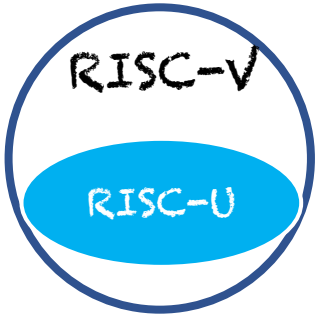
Selfie

Compiler

Symbolicaly execute

# Platform:



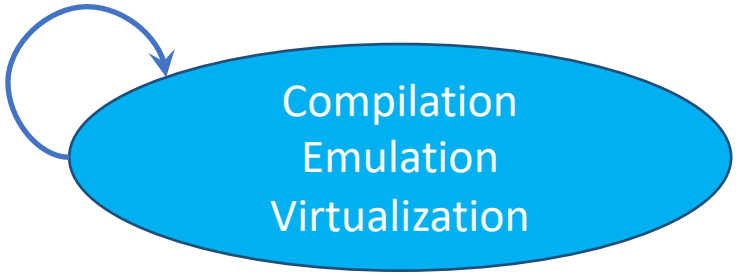




uint64_t
uint64_t*

Simplicity

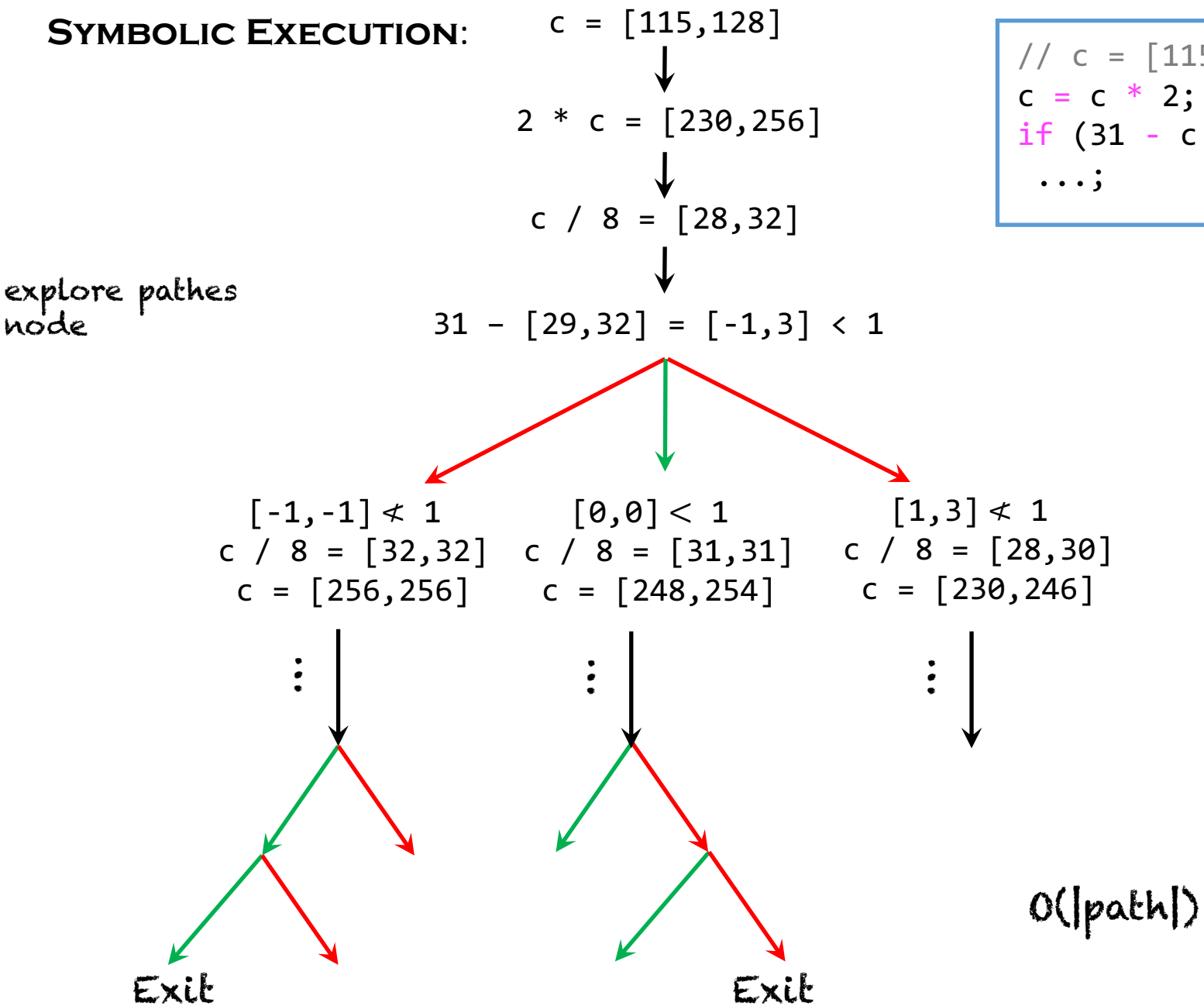
14 instruction

Self-referentiality



## Symbolic Execution:

```
// c = [115,128]
c = c * 2;
if (31 - c / 8 < 1)
 ...;
```



```
b = [0, 5];
c = b + 10;
n = 2 * c;
if (n <= 25)
  m = b + 1;
```

c = [10, 15]

n = [20, 30]

n = [20, 24], c = [10, 12], b = [0, 2]

b = [1, 6] (or) b = [1, 3]

Aliasing

## Goal:

Or part of the program

**Symbolic Analysis of a class of programs**

Sound
Incompleteness-aware
Efficient

What percentage?

Make a trade-off between completeness and performance for an interesting class of programs considering a simple numerical analysis

In particular for the code of Selfie

No external tools

0 <= 'd' - fuzz <= scanned_char <= 'd' + fuzz <= 255

```
uint64_t d = 15;

void main() {
  uint64_t f;
  f = 19;
}
```

# How to model machine behavior:

c1; ⇢ 8 bits
c2; ⇢ 8 bits

c1 in [60, 65];
c2 = c1 * 4;

c2 = {240, 244, 248, 252, 0, 4} mod $2^8$

step

wrap-around

How to represent values?

## Abstract representation:

As we are executing the machine code, we have to model the behavior of machine

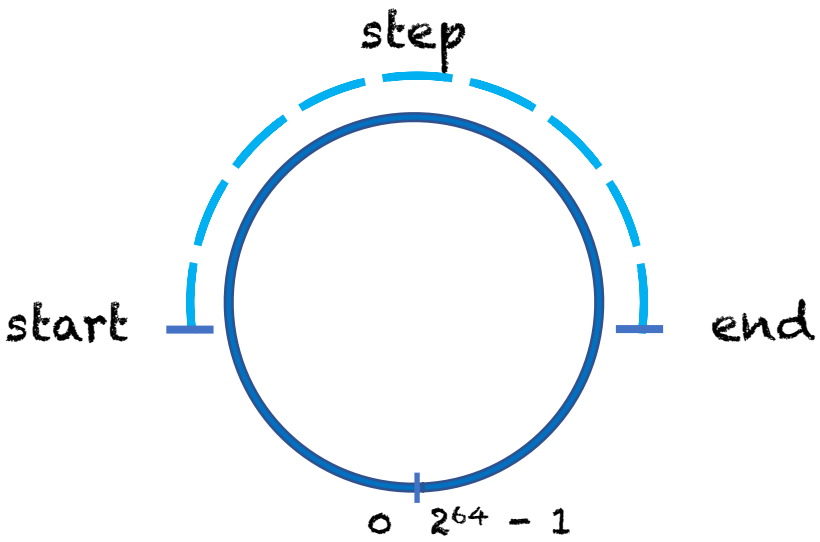
Arithmetic, Bitwise, Control operations

64-bit fixed precision integers

What is the simplest abstract domain to represent values?
Integer Interval with wrap-around semantics?

# Numerical Analysis:

## Modular Integer Interval Domain

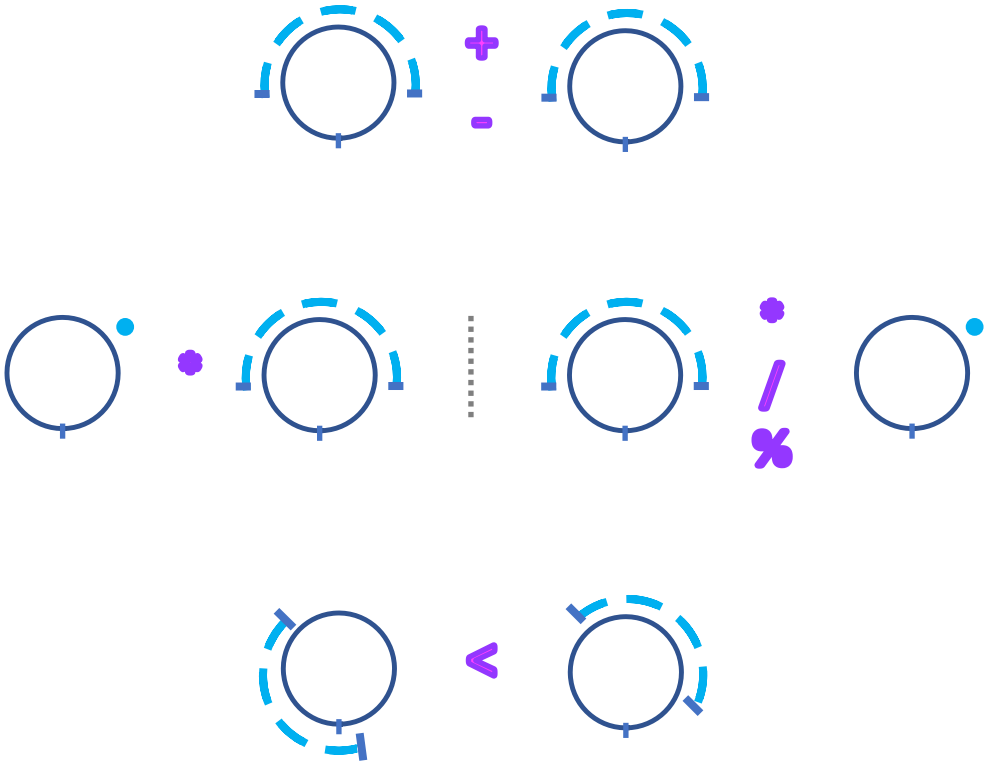



<start, end, step>

wrap-around interval

c1 = <60, 65, 1>
c2 = {240, 244, 248, 252, 0, 4} mod $2^8$ = <240, 4, 4>

## Semantics of the operations:

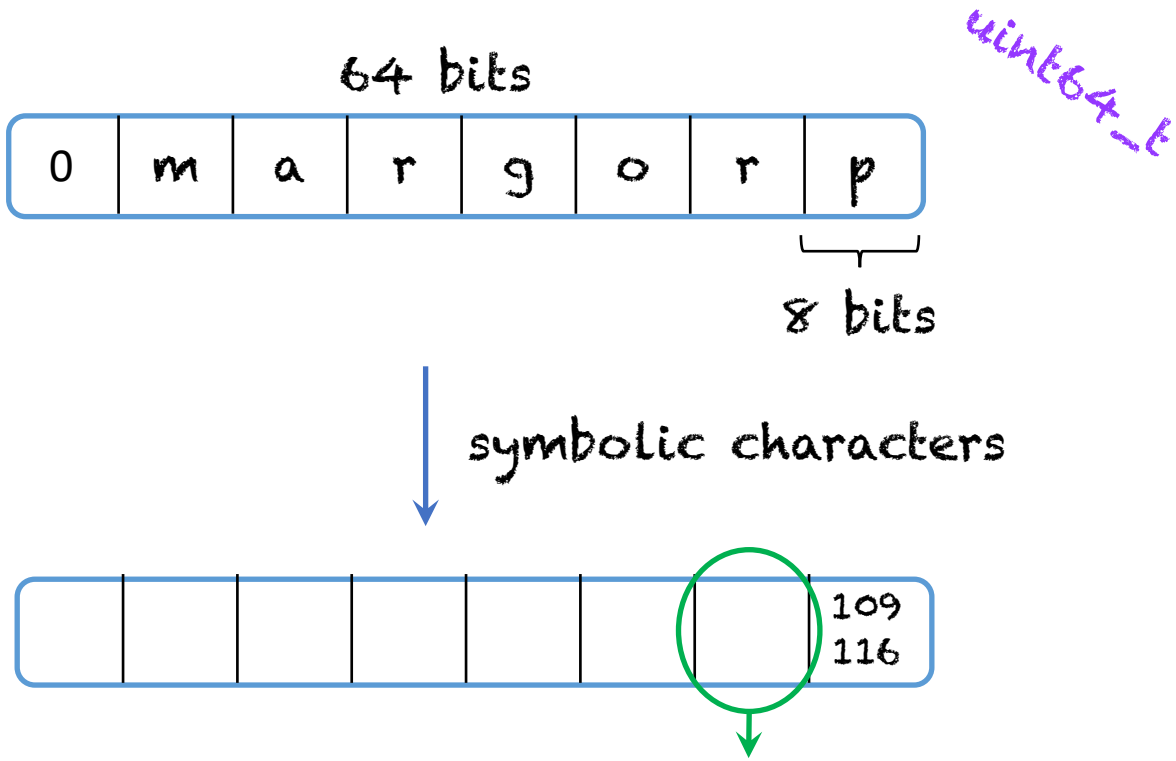



1. Is it possible to over-approximate all possible values?
2. Is it possible to represent all guaranteed values?

using our abstract domain

## Constraint type:

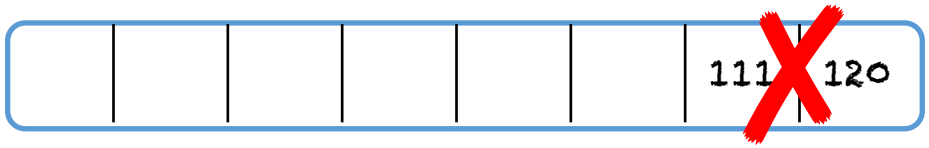
s = "program";




$0 < 111 <= char <= 118 < 255$

1 constraint

$<111*2^8+109,\ 118*2^8+116,\ 1>$




$111*2^8+120$

8 characters
8 constraints

Infer type of constraints: from byte-level to double word

```
uint64_t* n = "19"  →  n = 0 * 10 + 1;
                       n = 1 * 10 + 9;  →  uint64_t n = 19
```

0<= c1 <=9

0<= c2 <=9

<0, 90, 10> + <0, 9, 1> = <0, 99, 1>

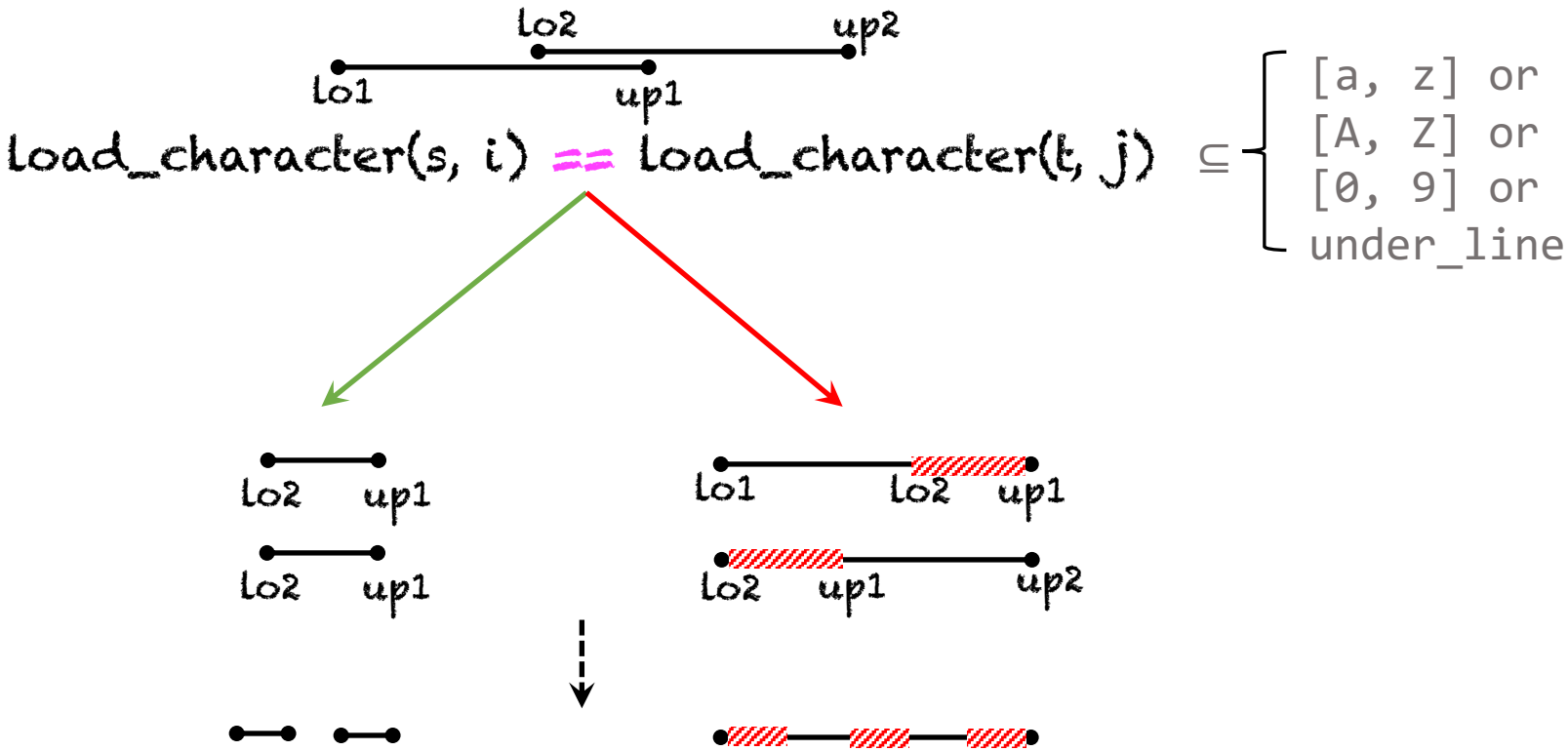
```
uint64_t* n = "19"  →  n = 0 * 10 + 1;
                       n = 1 * 10 + 9;  →  uint64_t n = 19
```

0<= c1 <=3

7<= c2 <=9

<0, 30, 10> + <6, 9, 1> ⊂ <6, 39, 1>

# Relations:


lo2
up2
lo1
up1
load_character(s, i) == load_character(t, j) ⊆
[a, z] or
[A, Z] or
[0, 9] or
under_line
lo2
up1
lo2
up1
lo1
lo2
up1
lo2
up1
up2

took the code of the compiler

How far we can go considering a simple abstract domain?

Performance ⟷ Completeness

O(|path|)?

There was cases where cannot be handled